Analyzing Crop Production Across India

India, an agrarian economy, has witnessed significant transformations in its agricultural patterns due to factors such as urbanization, climate change, and technological advances. Analyzing these shifts at the district level provides insights into food security, resource allocation, and sustainable farming practices.

However, while existing studies extensively explore macro-level trends (state or national scale), granular district-level insights over extended timeframes (1997–2023) remain underexplored. This project aims at trying to explore this segment in Indian crop production

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import scipy.stats as stats
import seaborn as sns
import geopandas as gpd

import folium
from folium import plugins
import branca.colormap as cm

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler,
StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.tree import plot_tree
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score

from sklearn.decomposition import PCA

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

APY = pd.read_csv("APY.csv")
rainfall = pd.read_csv("rainfall.csv")
temp = pd.read_csv("temperature.csv")
agri_pricing = pd.read_csv("agr_csv2.csv")
india_map = gpd.read_file('States/India_State_Boundary.shp')
```

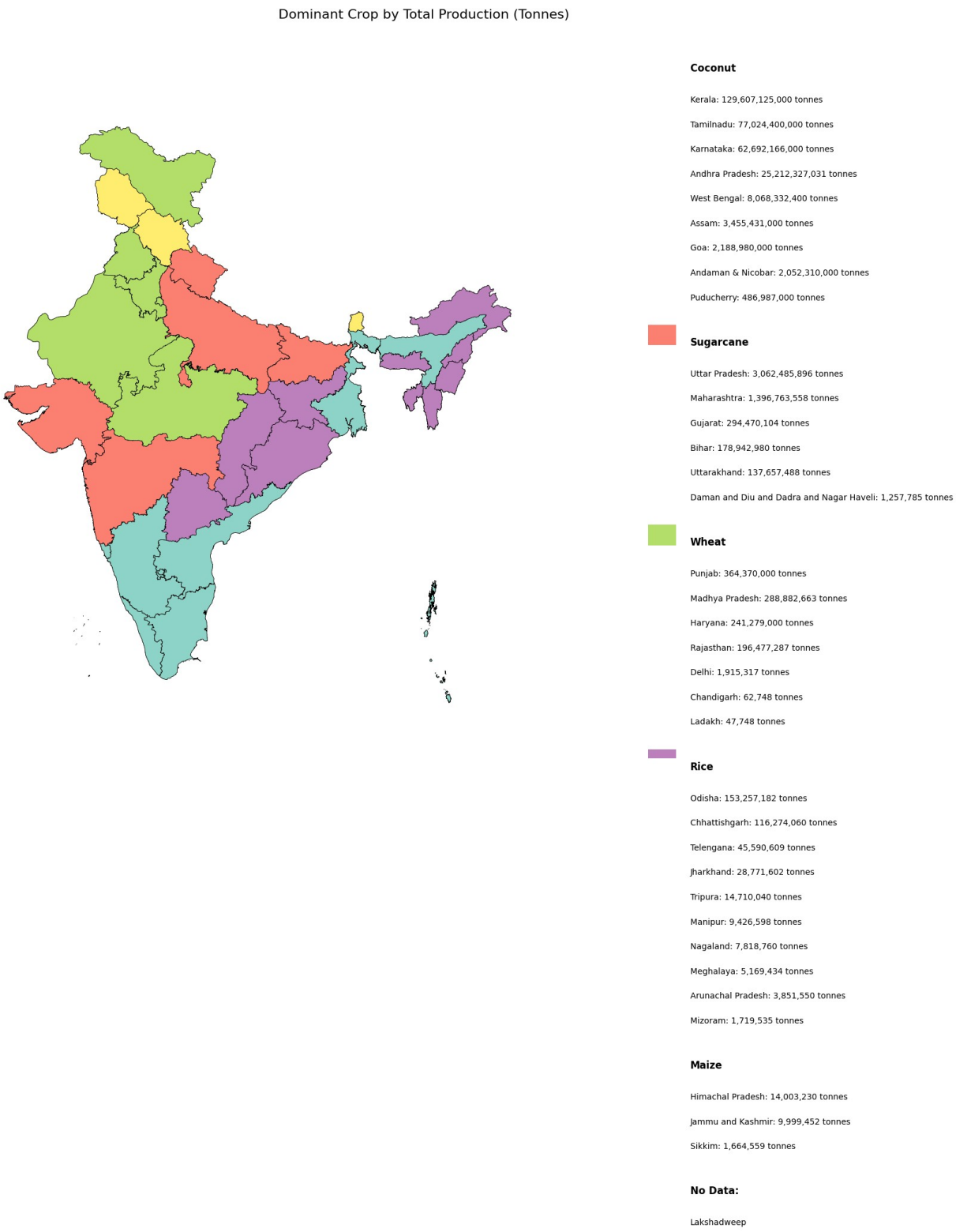
```
APY_dict = {
    'APY': APY,
    'rainfall': rainfall,
    'temp': temp,
    'agri_pricing': agri_pricing
}

for i, j in APY_dict.items():
    j.dropna(axis=0)
    print(f"{i}'s shape is {j.shape}")
```

```
APY's shape is (345336, 8)
rainfall's shape is (58, 18)
temp's shape is (58, 6)
agri_pricing's shape is (2238, 9)
```

```
# mapping the statenames in APY to shapefile so that there's
consistency
STATE_NAME_MAPPING = {
    # APY dataset names -> Shapefile names
    'Andaman and Nicobar Island': 'Andaman & Nicobar',
    'Chhattisgarh': 'Chhattishgarh',
    'Dadra and Nagar Haveli': 'Daman and Diu and Dadra and Nagar
Haveli',
    'THE DADRA AND NAGAR HAVELI': 'Daman and Diu and Dadra and Nagar
Haveli',
    'Daman and Diu': 'Daman and Diu and Dadra and Nagar Haveli',
    'Tamil Nadu': 'Tamilnadu',
    'Telangana': 'Telengana',
    'Laddak': 'Ladakh',
    'CHANDIGARH': 'Chandigarh'
}

# Calculate the Total Production of each state and plot it

APY_standardized = APY.copy()
APY_standardized['State'] =
APY_standardized['State'].replace(STATE_NAME_MAPPING)

total_production = APY_standardized.groupby(['State', 'Crop'])
['Production'].sum().reset_index()

dominant_crops = total_production.loc[
    total_production.groupby('State')['Production'].idxmax()
]

dominant_crops = dominant_crops.sort_values('Production',
ascending=False)

fig = plt.figure(figsize=(20, 15))
gs = plt.GridSpec(1, 2, width_ratios=[2, 1])
```

```
ax_map = fig.add_subplot(gs[0])
ax_legend = fig.add_subplot(gs[1])

# Create color mapping
unique_crops = dominant_crops['Crop'].unique()
colors = plt.cm.Set3(np.linspace(0, 1, len(unique_crops)))
crop_colors = dict(zip(unique_crops, colors))

# Merge with geographic data
merged_data = india_map.merge(
    dominant_crops,
    how='left',
    left_on='State_Name',
    right_on='State'
)

# Plot base map
merged_data.plot(
    ax=ax_map,
    color='lightgray',
    edgecolor='black',
    linewidth=0.5
)

# Plot states with data
for crop in unique_crops:
    crop_states = merged_data[merged_data['Crop'] == crop]
    if not crop_states.empty:
        crop_states.plot(
            ax=ax_map,
            color=crop_colors[crop],
            edgecolor='black',
            linewidth=0.5
        )

# Create detailed legend
ax_legend.axis('off')
y_position = 1
line_height = 0.03

# Group states by crop
for crop in unique_crops:
    crop_data = dominant_crops[dominant_crops['Crop'] == crop]

    # Add crop header
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                           facecolor=crop_colors[crop]))
    ax_legend.text(0.25, y_position, f'{crop}', fontsize=12,
fontweight='bold')
```

```
    y_position -= line_height * 1.5

    # Add states for this crop
    for _, row in crop_data.iterrows():
        text = f"{row['State']}: {row['Production']:,.0f} tonnes"
        ax_legend.text(0.25, y_position, text, fontsize=10)
        y_position -= line_height * 1.2

    y_position -= line_height  # Extra space between crops

# Add "No Data" entry
if len(merged_data[merged_data['Crop'].isna()]) > 0:
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                           facecolor='lightgray'))
    ax_legend.text(0.25, y_position, 'No Data:', fontsize=12,
fontweight='bold')
    y_position -= line_height * 1.5

    no_data_states = merged_data[merged_data['Crop'].isna()]
['State_Name'].unique()
    for state in sorted(no_data_states):
        ax_legend.text(0.25, y_position, state, fontsize=10)
        y_position -= line_height * 1.2

plt.suptitle('Dominant Crop by Total Production (Tonnes)', y=0.95,
fontsize=16)
ax_map.axis('off')
plt.show()
```

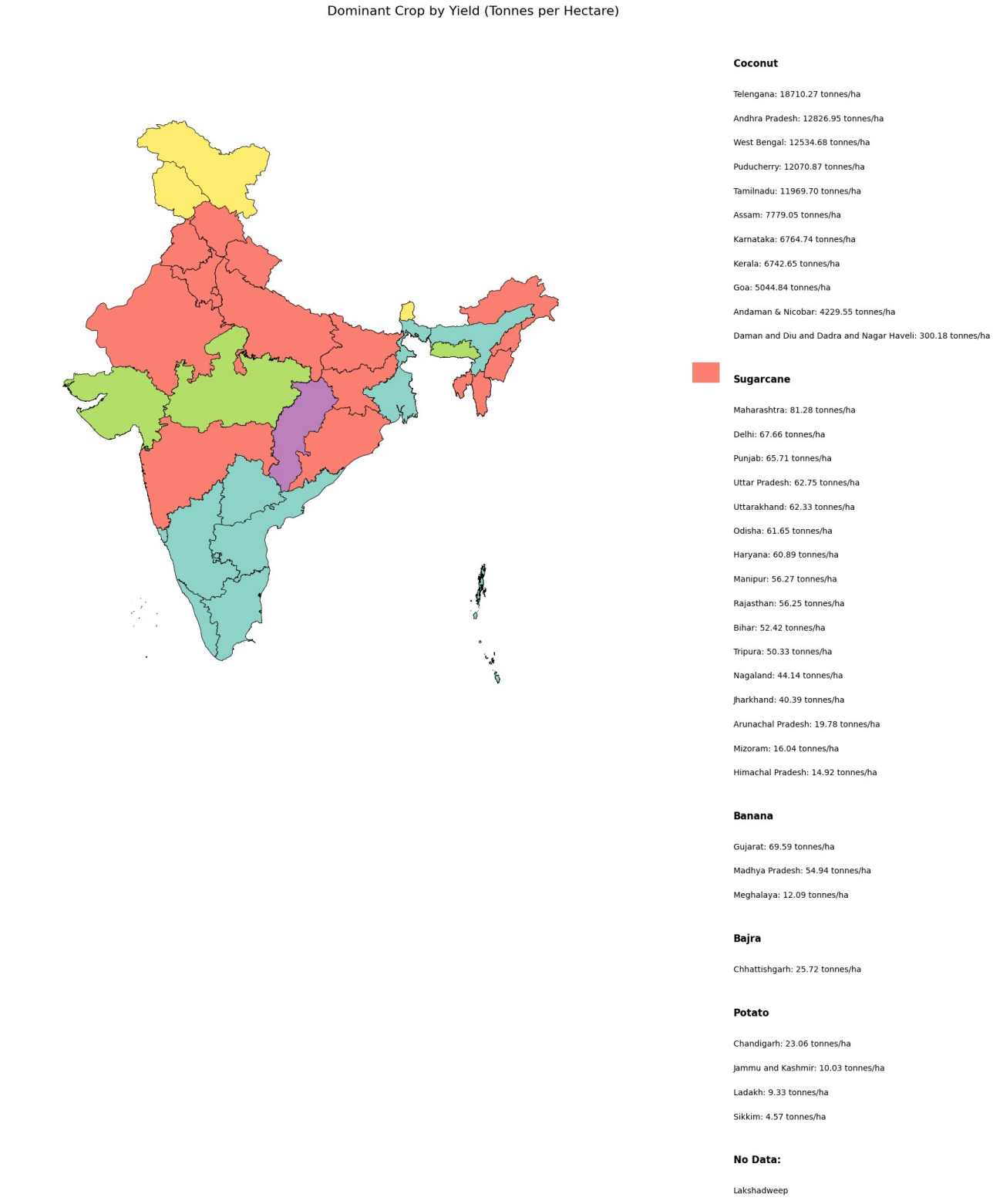



```
# Calculate the Total Production/Hectare of each state and plot it

APY_standardized = APY.copy()
```

```
APY_standardized['State'] =
APY_standardized['State'].replace(STATE_NAME_MAPPING)

# Calculate yield for each state-crop
state_yields = APY_standardized.groupby(['State', 'Crop']).agg({
    'Production': 'sum',
    'Area ': 'sum'
}).reset_index()
state_yields['Yield'] = state_yields['Production'] /
state_yields['Area ']

# Find crop with maximum yield for each state
dominant_yields = state_yields.loc[
    state_yields.groupby('State')['Yield'].idxmax()
]

# Sort states by yield for legend
dominant_yields = dominant_yields.sort_values('Yield',
ascending=False)

# Create visualization
fig = plt.figure(figsize=(20, 15))
gs = plt.GridSpec(1, 2, width_ratios=[2, 1])  # Adjust ratio for map
and legend
ax_map = fig.add_subplot(gs[0])
ax_legend = fig.add_subplot(gs[1])

# Create color mapping
unique_crops = dominant_yields['Crop'].unique()
colors = plt.cm.Set3(np.linspace(0, 1, len(unique_crops)))
crop_colors = dict(zip(unique_crops, colors))

# Merge with geographic data
merged_data = india_map.merge(
    dominant_yields,
    how='left',
    left_on='State_Name',
    right_on='State'
)

# Plot base map
merged_data.plot(
    ax=ax_map,
    color='lightgray',
    edgecolor='black',
    linewidth=0.5
)

# Plot states with data
for crop in unique_crops:
```

```
    crop_states = merged_data[merged_data['Crop'] == crop]
    if not crop_states.empty:
        crop_states.plot(
            ax=ax_map,
            color=crop_colors[crop],
            edgecolor='black',
            linewidth=0.5
        )

# Create detailed legend
ax_legend.axis('off')
y_position = 1
line_height = 0.03

# Group states by crop
for crop in unique_crops:
    crop_data = dominant_yields[dominant_yields['Crop'] == crop]

    # Add crop header
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                           facecolor=crop_colors[crop]))
    ax_legend.text(0.25, y_position, f'{crop}', fontsize=12,
fontweight='bold')
    y_position -= line_height * 1.5

    # Add states for this crop
    for _, row in crop_data.iterrows():
        text = f"{row['State']}: {row['Yield']:.2f} tonnes/ha"
        ax_legend.text(0.25, y_position, text, fontsize=10)
        y_position -= line_height * 1.2

    y_position -= line_height  # Extra space between crops

# Add "No Data" entry
if len(merged_data[merged_data['Crop'].isna()]) > 0:
    ax_legend.add_patch(mpatches.Rectangle((0.1, y_position), 0.1,
line_height,
                                           facecolor='lightgray'))
    ax_legend.text(0.25, y_position, 'No Data:', fontsize=12,
fontweight='bold')
    y_position -= line_height * 1.5

    no_data_states = merged_data[merged_data['Crop'].isna()]
['State_Name'].unique()
    for state in sorted(no_data_states):
        ax_legend.text(0.25, y_position, state, fontsize=10)
        y_position -= line_height * 1.2

plt.suptitle('Dominant Crop by Yield (Tonnes per Hectare)', y=0.95,
```

```
fontsize=16)
ax_map.axis('off')
plt.show()
```



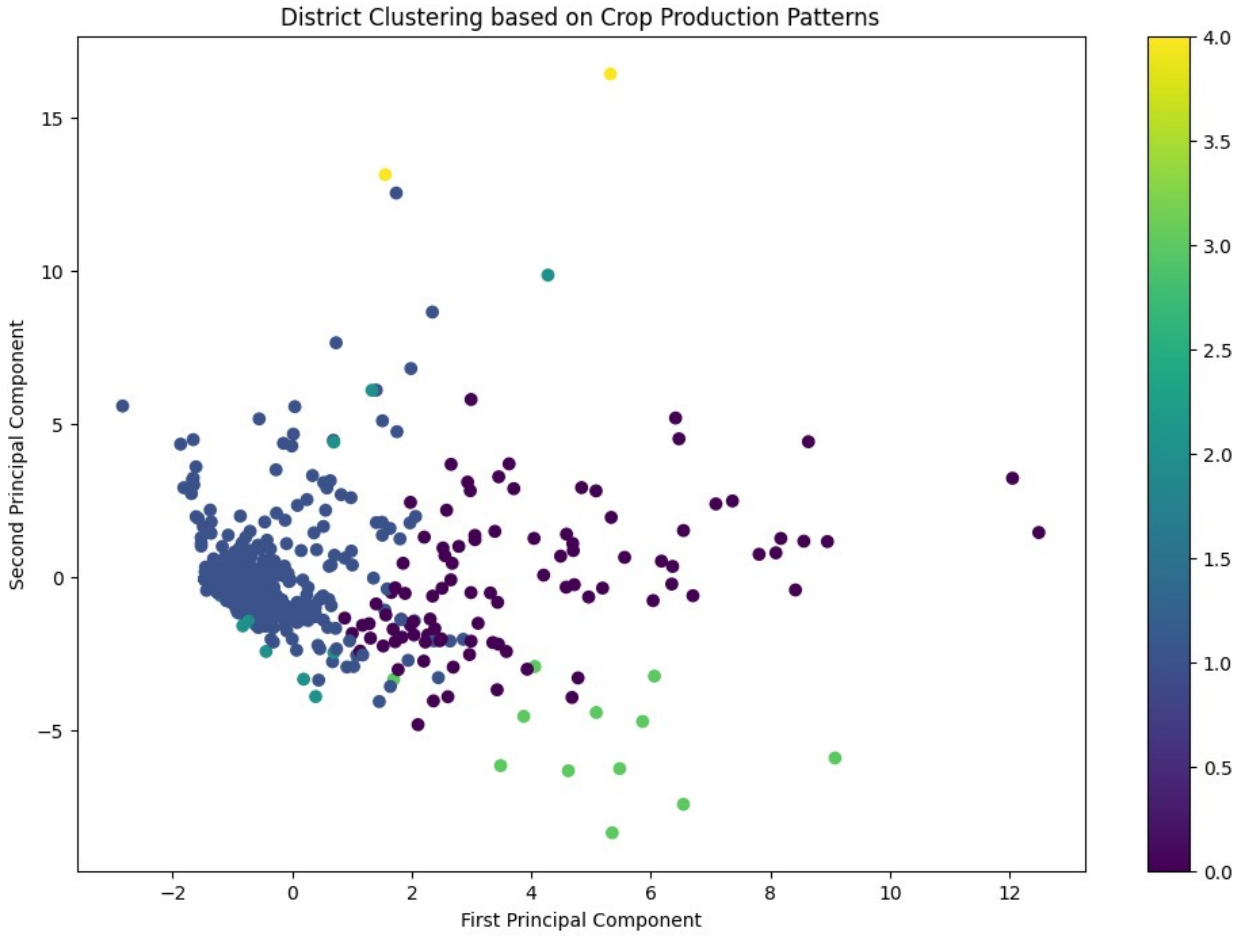
Implementing K-NN to figure out what parts of India grow what major crop

```
def data_ready(APY):

    district_crop_summary = APY.groupby(['District', 'Crop'])
['Production'].sum().reset_index()

    dominant_crops = district_crop_summary.loc[
        district_crop_summary.groupby('District')
['Production'].idxmax()
    ]

    # Print summary statistics
    print("\n=== Crop Distribution Summary ===")
    print("\nNumber of unique crops:", len(APY['Crop'].unique()))
    print("\nTop 10 most common crops:")
    print(APY['Crop'].value_counts().head(10))

    return district_crop_summary, dominant_crops
# data_ready(APY)

def analyze_crop_distribution(APY):
    # Prepare the data
    district_crop_summary, dominant_crops = data_ready(APY)

    # Create pivot table for district-crop production
    pivot_APY = district_crop_summary.pivot(
        index='District',
        columns='Crop',
        values='Production'
    ).fillna(0)

    # Scale the data
    scaler = StandardScaler()
    scaled_data = scaler.fit_transform(pivot_APY)

    # Apply K-means clustering
    n_clusters = 5
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(scaled_data)

    # Apply PCA for visualization
    pca = PCA(n_components=2)
    pca_result = pca.fit_transform(scaled_data)

    # Create visualization
    plt.figure(figsize=(12, 8))
    scatter = plt.scatter(pca_result[:, 0], pca_result[:, 1],
c=labels, cmap='viridis')
```

```
    plt.title('District Clustering based on Crop Production Patterns')
    plt.xlabel('First Principal Component')
    plt.ylabel('Second Principal Component')
    plt.colorbar(scatter)
    plt.show()

    # Analyze clusters
    cluster_APY = pd.DataFrame({
        'District': pivot_APY.index,
        'Cluster': labels
    })

    # For each cluster, find the most common crops
    print("\n=== Cluster Analysis ===")
    for cluster in range(n_clusters):
        cluster_districts = cluster_APY[cluster_APY['Cluster'] ==
cluster]['District']
        cluster_crops = district_crop_summary[
            district_crop_summary['District'].isin(cluster_districts)
        ]

        print(f"\nCluster {cluster} characteristics:")
        print("Number of districts:", len(cluster_districts))
        print("Top 5 crops by production:")
        print(cluster_crops.groupby('Crop')['Production']
              .sum()
              .sort_values(ascending=False)
              .head()
        )
        print("\nDistricts in this cluster:")
        print(cluster_districts.tolist())

    # Additional analysis of cluster characteristics
    cluster_APY['Dominant_Crop'] =
dominant_crops.set_index('District')['Crop']

    print("\n=== Cluster Summary ===")
    print("\nNumber of districts in each cluster:")
    print(cluster_APY['Cluster'].value_counts())

    print("\nDominant crops by cluster:")
    print(cluster_APY.groupby('Cluster')
['Dominant_Crop'].value_counts())

    return cluster_APY, dominant_crops

def create_detailed_district_report(district_name, df):
    """Create a detailed report for a specific district"""
    district_data = df[df['District'] == district_name]
    yearly_production = district_data.groupby(['Crop', 'Crop_Year'])
```

```
['Production'].sum().reset_index()

    average_yearly_production = yearly_production.groupby('Crop')
['Production'].mean()\
        .sort_values(ascending=False)\
        .head()

    print(f"\n=== Detailed Report for {district_name} ===")
    print("\nTop 5 crops by production:")
    print(average_yearly_production)

    print("\nSeasonal distribution:")
    print(district_data['Season'].value_counts())

    top_crop = district_data.groupby('Crop')
['Production'].sum().idxmax()
    yearly_trend = district_data[district_data['Crop'] ==
top_crop].groupby('Crop_Year')['Production'].sum()

    plt.figure(figsize=(10, 6))
    yearly_trend.plot(kind='line')
    plt.title(f'Production Trend of {top_crop} in {district_name}')
    plt.xlabel('Year')
    plt.ylabel('Production')
    plt.show()

def visualize_regional_patterns(cluster_df, dominant_crops):
    plt.figure(figsize=(15, 8))

    # Create a map of clusters
    sns.scatterplot(data=cluster_df, x='longitude', y='latitude',
hue='Cluster', style='Cluster')
    plt.title('Regional Crop Production Clusters')
    plt.xlabel('Longitude')
    plt.ylabel('Latitude')
    plt.show()

    # Create crop distribution heatmap
    pivot_table = pd.crosstab(dominant_crops['District'],
dominant_crops['Crop'])
    plt.figure(figsize=(15, 10))
    sns.heatmap(pivot_table, cmap='YlOrRd', cbar_kws={'label': 'Number
of Districts'})
    plt.title('Crop Distribution Across Districts')
    plt.xticks(rotation=45)
    plt.show()

cluster_df, dominant_crops = analyze_crop_distribution(APY)
```

```
=== Crop Distribution Summary ===

Number of unique crops: 56

Top 10 most common crops:
Rice                 21611
Maize                20513
Moong(Green Gram)    15139
Urad                 14581
Sesamum              13049
Groundnut            12586
Wheat                11220
Rapeseed &Mustard    11034
Sugarcane            10942
Arhar/Tur            10885
Name: Crop, dtype: int64

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/sklearn/cluster/_kmeans.py:870: FutureWarning: The
default value of `n_init` will change from 10 to 'auto' in 1.4. Set
the value of `n_init` explicitly to suppress the warning
  warnings.warn(
```




```
=== Cluster Analysis ===

Cluster 0 characteristics:
Number of districts: 98
Top 5 crops by production:
Crop
Coconut          8.404252e+09
Sugarcane        2.119517e+09
Wheat            3.533353e+08
Cotton(lint)     2.926095e+08
Rice             2.285058e+08
Name: Production, dtype: float64

Districts in this cluster:
['adilabad', 'ahmednagar', 'akola', 'amravati', 'amreli', 'anantapur',
'ashoknagar', 'aurangabad', 'bagalkote', 'balaghat', 'ballari',
'banda', 'baran', 'beed', 'belagavi', 'betul', 'bhavnagar', 'bidar',
'buldhana', 'chandrapur', 'chhatarpur', 'chhindwara', 'chitradurga',
'chittorgarh', 'damoh', 'dewas', 'dhar', 'dharwad', 'dhule', 'gadag',
'guna', 'guntur', 'hamirpur', 'haveri', 'hingoli', 'hoshangabad',
```

```
'indore', 'jabalpur', 'jalaun', 'jalgaon', 'jalna', 'jamnagar',
'jhalawar', 'jhansi', 'junagadh', 'kadapa', 'kalaburagi',
'karimnagar', 'khargone', 'kolhapur', 'koppal', 'kota', 'krishna',
'kurnool', 'lalitpur', 'latur', 'mahbubnagar', 'mahoba', 'mandsaur',
'medak', 'nagpur', 'nanded', 'nandurbar', 'narsinghpur', 'nashik',
'neemuch', 'nizamabad', 'osmanabad', 'panna', 'parbhani', 'prakasam',
'pune', 'raichur', 'raisen', 'rajgarh', 'rajkot', 'ratlam', 'rewa',
'sagar', 'sangli', 'satara', 'satna', 'sehore', 'seoni', 'shajapur',
'shivpuri', 'sidhi', 'solapur', 'surendranagar', 'tikamgarh',
'ujjain', 'vadodara', 'vidisha', 'vijayapura', 'warangal', 'wardha',
'washim', 'yavatmal']

Cluster 1 characteristics:
Number of districts: 581
Top 5 crops by production:
Crop
Coconut      2.683292e+11
Sugarcane    4.971582e+09
Rice         1.664648e+09
Wheat        1.549058e+09
Potato       3.983404e+08
Name: Production, dtype: float64

Districts in this cluster:
['agar malwa', 'agra', 'ahmadabad', 'aizawl', 'ajmer', 'alappuzha',
'aligarh', 'alipurduar', 'alirajpur', 'allahabad', 'almora', 'ambala',
'ambedkar nagar', 'amethi', 'amritsar', 'amroha', 'anand', 'anantnag',
'anjaw', 'anugul', 'anuppur', 'araria', 'aravalli', 'ariyalur',
'arwal', 'auraiya', 'azamgarh', 'badgam', 'bageshwar', 'baghpat',
'bahraich', 'baksa', 'balangir', 'baleshwar', 'ballia', 'balod',
'baloda bazar', 'balrampur', 'bandipora', 'bangalore rural', 'banka',
'banswara', 'barabanki', 'baramulla', 'bareilly', 'bargarh',
'barnala', 'barpeta', 'barwani', 'bastar', 'basti', 'bathinda',
'begusarai', 'bemetara', 'bengaluru urban', 'bhadradri', 'bhadrak',
'bhagalpur', 'bhandara', 'bharatpur', 'bharuch', 'bhilwara', 'bhind',
'bhiwani', 'bhojpur', 'bhopal', 'bijapur', 'bijnor', 'bilaspur',
'bishnupur', 'biswanath', 'bokaro', 'bongaigaon', 'botad', 'boudh',
'budaun', 'bulandshahr', 'bundi', 'burhanpur', 'buxar', 'cachar',
'chamarajanagara', 'chamba', 'chamoli', 'champawat', 'champhai',
'chandauli', 'chandel', 'chandigarh', 'changlang', 'charaideo',
'charki dadri', 'chatra', 'chengalpattu', 'chennai', 'chhotaudepur',
'chikkaballapura', 'chikkamagaluru', 'chirang', 'chitrakoot',
'chittoor', 'churachandpur', 'coimbatore', 'cuddalore', 'cuttack',
'dadra and nagar haveli', 'dakshina kannada', 'daman', 'dang',
'dantewada', 'darbhanga', 'darjeeling', 'darrang', 'datia', 'dausa',
'davangere', 'dehradun', 'delhi_total', 'deogarh', 'deoghar',
'deoria', 'devbhumi dwarka', 'dhalai', 'dhamtari', 'dhanbad',
'dharmapuri', 'dhemaji', 'dhenkanal', 'dholpur', 'dhubri', 'dibang
valley', 'dibrugarh', 'dima hasao', 'dimapur', 'dinajpur dakshin',
```

'dindigul', 'dindori', 'diu', 'doda', 'dohad', 'dumka', 'dungarpur',
'durg', 'east district', 'east garo hills', 'east jaintia hills',
'east kameng', 'east khasi hills', 'east siang', 'east singhbum',
'ernakulam', 'erode', 'etah', 'etawah', 'faizabad', 'faridabad',
'faridkot', 'farrukhabad', 'fatehabad', 'fatehgarh sahib', 'fatehpur',
'fazilka', 'firozabad', 'firozepur', 'gadchiroli', 'gajapati',
'ganderbal', 'gandhinagar', 'ganjam', 'garhwa', 'gariyaband',
'gaurella-pendra-marwahi', 'gautam buddha nagar', 'gaya', 'ghaziabad',
'ghazipur', 'gir somnath', 'giridih', 'goalpara', 'godda', 'golaghat',
'gomati', 'gonda', 'gondia', 'gopalganj', 'gorakhpur', 'gumla',
'gurdaspur', 'gurgaon', 'gwalior', 'hailakandi', 'hanumakonda',
'hapur', 'harda', 'hardoi', 'haridwar', 'hathras', 'hazaribagh',
'hisar', 'hojai', 'hoshiarpur', 'howrah', 'hyderabad', 'idukki',
'imphal east', 'imphal west', 'jagatsinghapur', 'jagitial',
'jaisalmer', 'jajapur', 'jalandhar', 'jalore', 'jalpaiguri', 'jammu',
'jamtara', 'jamui', 'jangoan', 'janjgir-champa', 'jashpur', 'jaunpur',
'jayashankar', 'jehanabad', 'jhabua', 'jhajjar', 'jhargram',
'jharsuguda', 'jind', 'jogulamba', 'jorhat', 'kabirdham', 'kachchh',
'kaimur (bhabua)', 'kaithal', 'kalahandi', 'kalimpong',
'kallakurichi', 'kamareddy', 'kamle', 'kamrup', 'kamrup metro',
'kanchipuram', 'kandhamal', 'kangra', 'kanker', 'kannauj',
'kanniyakumari', 'kannur', 'kanpur dehat', 'kanpur nagar',
'kapurthala', 'karaikal', 'karauli', 'karbi anglong', 'kargil',
'karimganj', 'karnal', 'karur', 'kasaragod', 'kasganj', 'kathua',
'katihar', 'katni', 'kaushambi', 'kendrapara', 'kendujhar',
'khagaria', 'khammam', 'khandwa', 'kheda', 'kheri', 'khordha',
'khowai', 'khunti', 'kinnaur', 'kiphire', 'kishanganj', 'kishtwar',
'kodagu', 'koderma', 'kohima', 'kokrajhar', 'kolar', 'kolasib',
'kollam', 'komaram bheem asifabad', 'kondagaon', 'koraput', 'korba',
'korea', 'kottayam', 'kozhikode', 'kra daadi', 'krishnagiri',
'kulgam', 'kullu', 'kupwara', 'kurukshetra', 'kurung kumey', 'kushi
nagar', 'lahul and spiti', 'lakhimpur', 'lakhisarai', 'latehar',
'lawngtlai', 'leh ladakh', 'leparada', 'lohardaga', 'lohit',
'longding', 'longleng', 'lower dibang valley', 'lower siang', 'lower
subansiri', 'lucknow', 'ludhiana', 'lunglei', 'madhepura',
'madhubani', 'madurai', 'mahabubabad', 'maharajganj', 'mahasamund',
'mahe', 'mahendragarh', 'mahesana', 'mahisagar', 'mainpuri', 'majuli',
'malappuram', 'maldah', 'malkangiri', 'mamit', 'mancherial', 'mandi',
'mandla', 'mandya', 'mansa', 'marigaon', 'mathura', 'mau',
'mayurbhanj', 'medchal malkajgiri', 'meerut', 'mewat', 'mirzapur',
'moga', 'mokokchung', 'mon', 'moradabad', 'morbi', 'morena',
'muktsar', 'mulugu', 'mumbai', 'mumbai suburban', 'mungeli', 'munger',
'muzaffarnagar', 'muzaffarpur', 'nabarangpur', 'nagaon',
'nagapattinam', 'nagarkurnool', 'nainital', 'nalanda', 'nalbari',
'nalgonda', 'namakkal', 'namsai', 'narayanapet', 'narayanpur',
'narmada', 'navsari', 'nawada', 'nayagarh', 'nicobars', 'nirmal',
'niwari', 'north and middle andaman', 'north district', 'north garo
hills', 'north goa', 'north tripura', 'nuapada', 'pakke kessang',
'pakur', 'palakkad', 'palamu', 'palghar', 'pali', 'palwal', 'panch

```
mahals', 'panchkula', 'panipat', 'papum pare', 'paraganas south',
'paschim bardhaman', 'pashchim champaran', 'patan', 'pathanamthitta',
'pathankot', 'patiala', 'patna', 'pauri garhwal', 'peddapalli',
'perambalur', 'peren', 'phek', 'pilibhit', 'pithoragarh',
'pondicherry', 'poonch', 'porbandar', 'pratapgarh', 'pudukkottai',
'pulwama', 'purbi champaran', 'puri', 'purnia', 'purulia', 'rae
bareli', 'raigad', 'raigarh', 'raipur', 'rajanna', 'rajauri',
'rajnandgaon', 'rajsamand', 'ramanagara', 'ramanathapuram', 'ramban',
'ramgarh', 'rampur', 'ranchi', 'rangareddi', 'ranipet', 'ratnagiri',
'rayagada', 'reasi', 'rewari', 'ri bhoi', 'rohtak', 'rohtas', 'rudra
prayag', 'rupnagar', 's nagar', 'sabar kantha', 'saharanpur',
'saharsa', 'sahebganj', 'saiha', 'salem', 'samastipur', 'samba',
'sambalpur', 'sambhal', 'sangareddy', 'sangrur', 'sant kabeer nagar',
'sant ravidas nagar', 'saraikela kharsawan', 'saran', 'sawai
madhopur', 'senapati', 'sepahijala', 'serchhip', 'shahdol', 'shahid
bhagat singh nagar', 'shahjahanpur', 'shamli', 'sheikhpura',
'sheohar', 'sheopur', 'shi yomi', 'shimla', 'shivamogga', 'shopian',
'shravasti', 'siang', 'siddharth nagar', 'siddipet', 'simdega',
'sindhudurg', 'singrauli', 'sirmaur', 'sirohi', 'sirsa', 'sitamarhi',
'sitapur', 'sivaganga', 'sivasagar', 'siwan', 'solan', 'sonbhadra',
'sonepur', 'sonipat', 'sonitpur', 'south andamans', 'south district',
'south garo hills', 'south goa', 'south salmara mancachar', 'south
tripura', 'south west garo hills', 'south west khasi hills', 'spsr
nellore', 'srikakulam', 'srinagar', 'sukma', 'sultanpur',
'sundargarh', 'supaul', 'surajpur', 'surat', 'surguja', 'suryapet',
'tamenglong', 'tapi', 'tarn taran', 'tawang', 'tehri garhwal',
'tenkasi', 'thane', 'thanjavur', 'the nilgiris', 'theni',
'thiruvallur', 'thiruvananthapuram', 'thiruvarur', 'thoubal',
'thrissur', 'tinsukia', 'tirap', 'tiruchirappalli', 'tirunelveli',
'tirupathur', 'tiruppur', 'tiruvannamalai', 'tonk', 'tuensang',
'tumakuru', 'tuticorin', 'udaipur', 'udalguri', 'udam singh nagar',
'udhampur', 'udupi', 'ukhrul', 'umaria', 'una', 'unakoti', 'unnao',
'upper siang', 'upper subansiri', 'uttar kashi', 'uttara kannada',
'vaishali', 'valsad', 'varanasi', 'vellore', 'vikarabad',
'villupuram', 'virudhunagar', 'visakhapatanam', 'wanaparthy',
'wayanad', 'west district', 'west garo hills', 'west jaintia hills',
'west kameng', 'west karbi anglong', 'west khasi hills', 'west siang',
'west singhbhum', 'west tripura', 'wokha', 'yadadri', 'yadagiri',
'yamunanagar', 'yanam', 'zunheboto']

Cluster 2 characteristics:
Number of districts: 14
Top 5 crops by production:
Crop
Coconut      2.368791e+10
Rice         3.306836e+08
Potato       1.841287e+08
Jute         1.586406e+08
Sugarcane    1.264099e+08
```

```
Name: Production, dtype: float64

Districts in this cluster:
['bankura', 'birbhum', 'coochbehar', 'dinajpur uttar', 'east
godavari', 'hooghly', 'medinipur east', 'medinipur west',
'murshidabad', 'nadia', 'paraganas north', 'purba bardhaman',
'vizianagaram', 'west godavari']

Cluster 3 characteristics:
Number of districts: 12
Top 5 crops by production:
Crop
Wheat                91046661.0
Bajra                63393277.0
Rapeseed &Mustard    36981566.0
Cotton(lint)         23697536.0
Guar seed            21664367.0
Name: Production, dtype: float64

Districts in this cluster:
['alwar', 'banas kantha', 'barmer', 'bikaner', 'churu', 'ganganagar',
'hanumangarh', 'jaipur', 'jhunjhunu', 'jodhpur', 'nagaur', 'sikar']

Cluster 4 characteristics:
Number of districts: 2
Top 5 crops by production:
Crop
Coconut      1.038339e+10
Sugarcane    2.812631e+07
Rice         1.033128e+07
Maize        6.048382e+06
Ragi         5.366618e+06
Name: Production, dtype: float64

Districts in this cluster:
['hassan', 'mysuru']

=== Cluster Summary ===

Number of districts in each cluster:
1    581
0     98
2     14
3     12
4      2
Name: Cluster, dtype: int64

Dominant crops by cluster:
Series([], Name: Dominant_Crop, dtype: int64)
```

What the Scatter Plot above means:

Clusters and Colors:

- Each dot represents a district
- The colors represent different clusters (from dark purple to yellow)
- Districts with the same color have similar crop production patterns

Position Meaning:

- The x-axis (First Principal Component) and y-axis (Second Principal Component) represent combined patterns of crop production
- Districts that are close together on the plot have similar crop production patterns
- Districts that are far apart have very different patterns

Key Observations:

- There's a dense blue cluster on the left side (-2, 0), suggesting many districts share a common agricultural pattern
- A scattered purple cluster spreads across the right side, indicating districts with more unique patterns
- Some outlier districts appear at the top of the plot (around y=15), suggesting very distinctive production patterns
- The spread along both axes shows significant variation in how districts produce crops

Practical Interpretation:

- The large blue cluster likely represents districts with "typical" or common crop production patterns
- The scattered points on the right and top might be districts specializing in specific crops or having unique agricultural profiles
- The gradual transition between clusters suggests some districts have hybrid production patterns

Now that we've covered what the scatter plot means, lets look at the geographical regions the clusters correspond to and majority of the crop

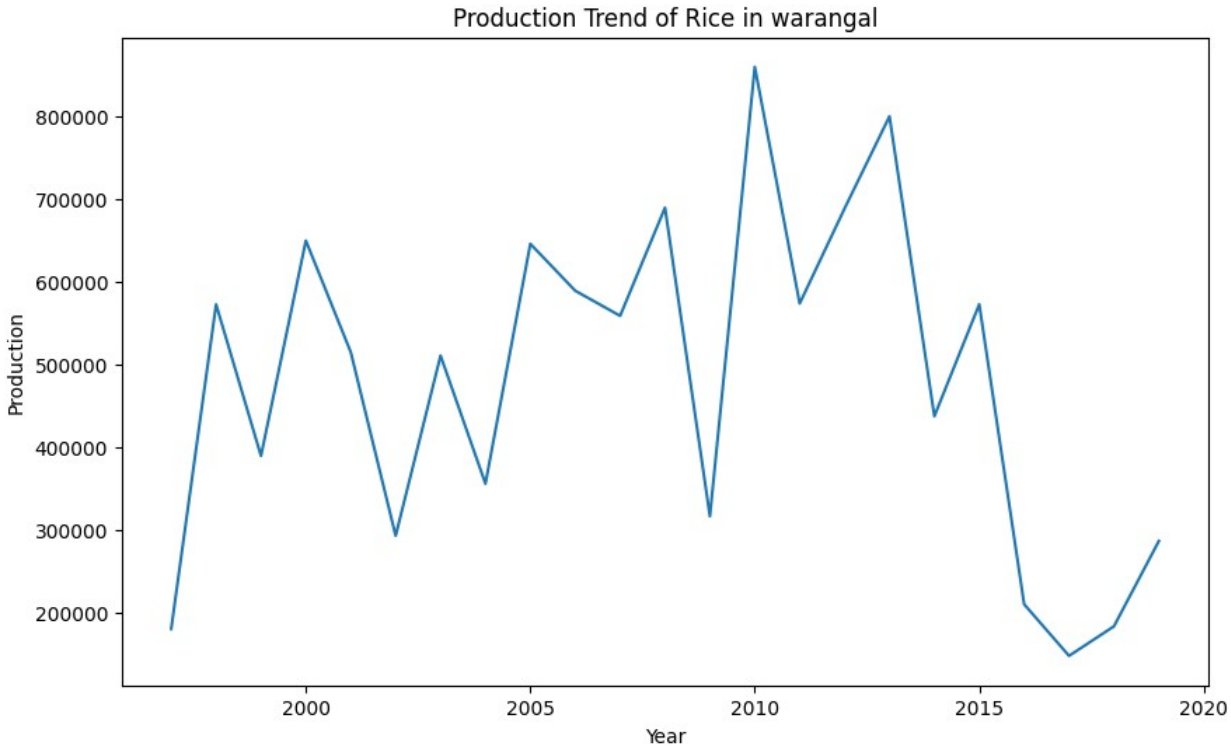
```
print("\nCluster assignments:")
print(cluster_df.head(5))

print("\nDominant crops:")
print(dominant_crops.head(5))
```

```
Cluster assignments:
     District  Cluster Dominant_Crop
0    adilabad        0           NaN
1  agar malwa        1           NaN
2        agra        1           NaN
3   ahmadabad        1           NaN
```

```
4  ahmednagar        0           NaN

Dominant crops:
       District          Crop   Production
6      adilabad  Cotton(lint)    9684901.0
70   agar malwa         Wheat    1474013.0
95         agra        Potato   27685472.0
111   ahmadabad  Cotton(lint)    6020981.0
161  ahmednagar     Sugarcane  151667127.0
```

Using the **create_detailed_district_report** function can retrive a lot of information about a district's production of the crops since 1961 on average

```
create_detailed_district_report('warangal', APY)


=== Detailed Report for warangal ===

Top 5 crops by production:
Crop
Rice             479475.086957
Cotton(lint)     383780.956522
Maize            283886.956522
Dry chillies      60934.652174
Groundnut         46095.521739
Name: Production, dtype: float64

Seasonal distribution:
Kharif         333
Rabi           326
Whole Year      71
Name: Season, dtype: int64
```



Implementing Time Analysis and forecasting now:

- First create time window, prepare climate features and the build the ltsm model

```
def prepare_climate_features(rainfall_df, temp_df):
    """
    Create relevant climate features from rainfall and temperature
data
    """
    try:
        # Create copy to avoid modifying original
        rainfall_features = rainfall_df.copy()
        temp_features = temp_df.copy()

        # Calculate seasonal rainfall
        rainfall_features['monsoon_rainfall'] =
rainfall_features[['JUN', 'JUL', 'AUG', 'SEP']].sum(axis=1)
        rainfall_features['pre_monsoon'] = rainfall_features[['MAR',
'APR', 'MAY']].sum(axis=1)
        rainfall_features['post_monsoon'] = rainfall_features[['OCT',
'NOV', 'DEC']].sum(axis=1)

        # Calculate temperature features - using actual column names
        if 'ANNUAL' in temp_features.columns:
            temp_features['annual_temp'] = temp_features['ANNUAL']
        elif 'JAN-FEB' in temp_features.columns:  # Using available
temperature periods
```

```
            temp_features['annual_temp'] = temp_features[['JAN-FEB',
'MAR-MAY', 'JUN-SEP', 'OCT-DEC']].mean(axis=1)

        temp_features['temp_variation'] = temp_features['JUN-SEP'] -
temp_features['annual_temp']

        # Merge climate data
        climate_data = pd.merge(rainfall_features, temp_features,
on='YEAR', suffixes=('_rain', '_temp'))

        return climate_data

    except Exception as e:
        print(f"Error in prepare_climate_features: {str(e)}")
        print("\nAvailable columns in rainfall_df:",
rainfall_df.columns.tolist())
        print("\nAvailable columns in temp_df:",
temp_df.columns.tolist())
        raise

def create_time_windows(data, window_size):
    """
    Create time windows for sequence prediction
    """
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:(i + window_size)])
        y.append(data[i + window_size])
    return np.array(X), np.array(y)

def build_lstm_model(input_shape, dropout_rate=0.2):
    """
    Build LSTM model for time series forecasting
    """
    model = Sequential([
        LSTM(64, activation='relu', input_shape=input_shape,
return_sequences=True),
        Dropout(dropout_rate),
        LSTM(32, activation='relu'),
        Dropout(dropout_rate),
        Dense(16, activation='relu'),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    return model
```

Now Analyze seasonality and create climate production dataset by:

1. Filtering production data for specific crop/district
2. Then aggregating production by year

3. After which we merge with climate features and return time-ordered dataset with production and climate variables

```
def analyze_seasonality(data, period=12):
    """
    Analyze seasonal patterns in time series data
    """
    result = stats.pearsonr(data[:-period], data[period:])
    lag_correlation = result[0]

    # Calculate seasonal indices
    seasonal_means = []
    for i in range(period):
        seasonal_means.append(np.mean(data[i::period]))
    seasonal_index = seasonal_means / np.mean(seasonal_means)

    return lag_correlation, seasonal_index

def create_climate_production_dataset(APY_df, rainfall_df, temp_df,
crop_name, district=None):
    """
    Create combined dataset of production and climate data
    """
    # Filter for specific crop and district
    if district:
        crop_data = APY_df[(APY_df['Crop'] == crop_name) &
                           (APY_df['District'] == district)].copy()
    else:
        crop_data = APY_df[APY_df['Crop'] == crop_name].copy()

    # Group by year
    yearly_production = crop_data.groupby('Crop_Year')
['Production'].sum().reset_index()
    yearly_production.columns = ['YEAR', 'Production']

    # Prepare climate data
    climate_data = prepare_climate_features(rainfall_df, temp_df)

    # Merge production and climate data
    combined_data = pd.merge(yearly_production, climate_data,
on='YEAR', how='inner')
    combined_data = combined_data.sort_values('YEAR')

    return combined_data

#Now we create a sequence data using time windows, obviosuly splitting
the data into a 80-20 split and training the ltsm model
def train_production_forecast_model(combined_data, feature_columns,
target_column='Production',
                                    window_size=5, train_split=0.8):
```

```
    """
    Train LSTM model for production forecasting
    """
    # Scale data
    scaler = MinMaxScaler()
    scaled_data = scaler.fit_transform(combined_data[feature_columns +
[target_column]])

    # Create sequences
    X, y = create_time_windows(scaled_data, window_size)

    # Split into train and test
    train_size = int(len(X) * train_split)
    X_train, X_test = X[:train_size], X[train_size:]
    y_train, y_test = y[:train_size], y[train_size:]

    # Build and train model
    model = build_lstm_model((window_size, len(feature_columns) + 1))
    history = model.fit(
        X_train, y_train,
        epochs=100,
        batch_size=32,
        validation_split=0.2,
        verbose=1
    )

    return model, scaler, history
```

1. Calculate correlations between each climate variable and production
2. Create scatter plots showing relationships
3. Return correlation coefficients

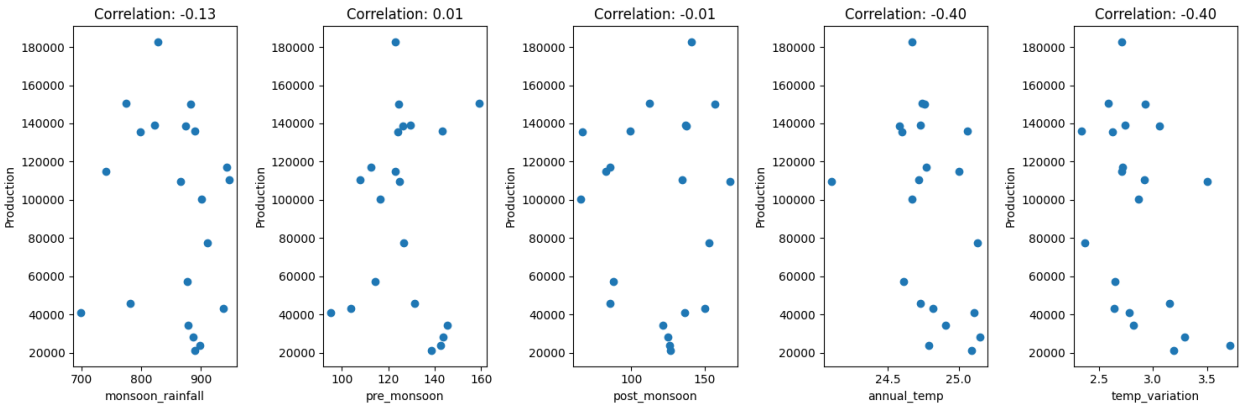
```
def analyze_climate_impact(combined_data):
    """
    Analyze relationship between climate variables and production
    """
    # Define climate columns based on what's available
    base_climate_cols = ['monsoon_rainfall', 'pre_monsoon',
'post_monsoon']
    temp_cols = ['annual_temp', 'temp_variation']

    # Check which columns exist in the data
    climate_cols = [col for col in base_climate_cols + temp_cols if
col in combined_data.columns]

    correlations = {}

    for col in climate_cols:
        correlation = stats.pearsonr(combined_data[col],
```

```
combined_data['Production'])
        correlations[col] = correlation[0]

    # Plot relationships
    n_cols = len(climate_cols)
    if n_cols > 0:
        fig_width = min(15, 5 * n_cols)
        plt.figure(figsize=(fig_width, 5))
        for i, col in enumerate(climate_cols, 1):
            plt.subplot(1, n_cols, i)
            plt.scatter(combined_data[col],
combined_data['Production'])
            plt.xlabel(col)
            plt.ylabel('Production')
            plt.title(f'Correlation: {correlations[col]:.2f}')
        plt.tight_layout()
        plt.show()

    return correlations

def forecast_future_production(model, scaler, last_sequence,
n_steps=10):
    """
    Generate future production forecasts
    """
    current_sequence = last_sequence.copy()
    forecasts = []

    for _ in range(n_steps):
        prediction = model.predict(current_sequence.reshape(1,
*current_sequence.shape), verbose=0)
        forecasts.append(prediction[0, 0])

        # Update sequence
        current_sequence = np.roll(current_sequence, -1, axis=0)
        current_sequence[-1] = prediction

    # Inverse transform predictions
    forecast_values = scaler.inverse_transform(
        np.column_stack([np.zeros((len(forecasts),
scaler.scale_.shape[0]-1)), forecasts])
    )[:, -1]

    return forecast_values

def plot_forecast_results(combined_data, forecast, crop_name,
district=None):
    """
    Plot historical data and forecasts
    """
```

```
    plt.figure(figsize=(15, 7))

    # Plot historical data
    plt.plot(combined_data['YEAR'], combined_data['Production'],
             label='Historical', linewidth=2)

    # Generate future years matching forecast length
    future_years = np.arange(
        combined_data['YEAR'].max() + 1,
        combined_data['YEAR'].max() + len(forecast) + 1
    )

    # Plot forecast
    plt.plot(future_years, forecast, '--', label='Forecast',
linewidth=2)

    # Add confidence intervals (using historical standard deviation)
    std_dev = combined_data['Production'].std()
    plt.fill_between(future_years,
                     forecast - std_dev,
                     forecast + std_dev,
                     alpha=0.2,
                     label='Confidence Interval')

    title = f'{crop_name} Production Forecast'
    if district:
        title += f' for {district}'
    plt.title(title)
    plt.xlabel('Year')
    plt.ylabel('Production')
    plt.grid(True, alpha=0.3)
    plt.legend()

    # Add value labels
    # for year, value in zip(future_years, forecast):
    #     plt.text(year, value, f'{value:.0f}',
    #              horizontalalignment='center',
    #              verticalalignment='bottom')

    plt.tight_layout()
    plt.show()

    # Print forecast values
    print("\nForecast Values:")
    print("Year\t\tPredicted Production")
    print("-" * 35)
    for year, value in zip(future_years, forecast):
        print(f"{year}\t\t{value:,.2f}")
```

As we can see in the previous few code blocks, the code format allows for data integration of climate dataset with the crop production and figure out their correlation, make an

```
def main_analysis1(APY_df, rainfall_df, temp_df, crop_name,
district=None, n_years=10):
    """
    Run complete time series analysis pipeline
    """
    print(f"\nAnalyzing {crop_name} production", f"in {district}" if
district else "across all districts")

    try:
        # Prepare dataset
        combined_data = create_climate_production_dataset(
            APY_df, rainfall_df, temp_df, crop_name, district
        )

        # Analyze seasonality
        lag_corr, seasonal_idx =
analyze_seasonality(combined_data['Production'].values)
        print(f"\nSeasonality Analysis:")
        print(f"Year-over-year correlation: {lag_corr:.2f}")

        # Analyze climate impact
        climate_correlations = analyze_climate_impact(combined_data)
        print("\nClimate Impact Correlations:")
        for var, corr in climate_correlations.items():
            print(f"{var}: {corr:.2f}")

    except Exception as e:
        print(f"\nError in analysis: {str(e)}")
        print("\nDebug information:")
        print("Temperature data columns:", temp_df.columns.tolist())
        print("Rainfall data columns:", rainfall_df.columns.tolist())
        raise

results = main_analysis1(APY, rainfall, temp, crop_name='Sugarcane',
district='guntur')


Analyzing Sugarcane production in guntur

Seasonality Analysis:
Year-over-year correlation: 0.69
```

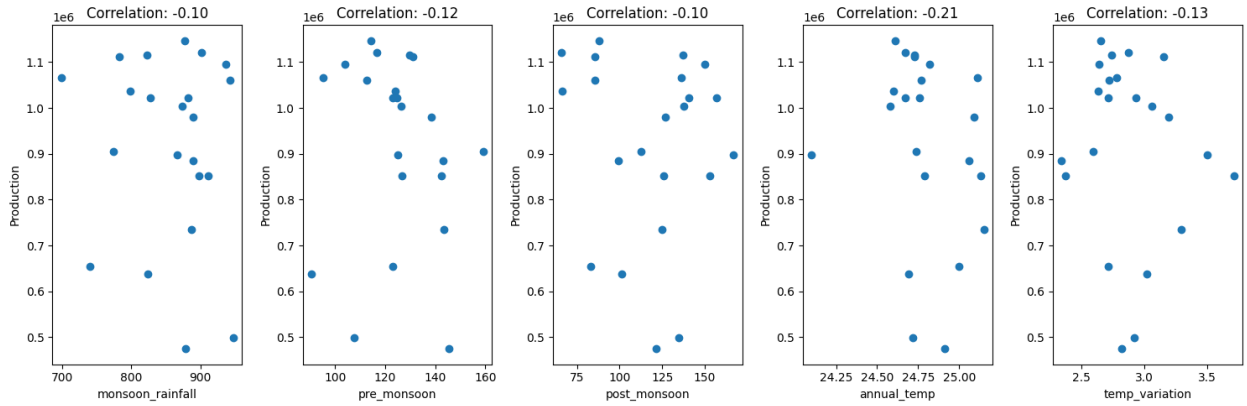



```
Climate Impact Correlations:
monsoon_rainfall: -0.13
pre_monsoon: 0.01
post_monsoon: -0.01
annual_temp: -0.40
temp_variation: -0.40
```

As we can see above, the code format allows for data integration of climate dataset with the crop production and figure out their correlation

```
def main_analysis2(APY_df, rainfall_df, temp_df, crop_name,
district=None, n_years=10):
    try:
        # Define variables:
        combined_data = create_climate_production_dataset(
            APY_df, rainfall_df, temp_df, crop_name, district
        )

        climate_correlations = analyze_climate_impact(combined_data)

        lag_corr, seasonal_idx =
analyze_seasonality(combined_data['Production'].values)

        # Train forecasting model
        feature_columns = list(climate_correlations.keys())
        model, scaler, history = train_production_forecast_model(
            combined_data, feature_columns
        )

        # Generate forecast
        last_sequence = scaler.transform(combined_data[feature_columns
+ ['Production']].iloc[-5:])
        forecast = forecast_future_production(model, scaler,
last_sequence, n_steps=n_years)
```

```
        # Plot results
        plot_forecast_results(combined_data, forecast, crop_name,
district)

        return {
            'model': model,
            'forecast': forecast,
            'climate_impact': climate_correlations,
            'seasonality': {'correlation': lag_corr, 'indices':
seasonal_idx},
            'data': combined_data
        }

    except Exception as e:
        print(f"\nError in analysis: {str(e)}")
        print("\nDebug information:")
        print("Temperature data columns:", temp_df.columns.tolist())
        print("Rainfall data columns:", rainfall_df.columns.tolist())
        raise

crop_prediction = main_analysis2(APY, rainfall, temp,
crop_name='Rice', district='guntur')
```




```
Epoch 1/100

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/
site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)

1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - loss: 0.3583 - mae: 0.5190 -
val_loss: 0.3165 - val_mae: 0.4915
Epoch 2/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 54ms/step - loss: 0.3492 - mae: 0.5102 -
```

```
val_loss: 0.3064 - val_mae: 0.4822
Epoch 3/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.3371 - mae: 0.4997 -
val_loss: 0.2983 - val_mae: 0.4746
Epoch 4/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.3248 - mae: 0.4892 -
val_loss: 0.2908 - val_mae: 0.4674
Epoch 5/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.3188 - mae: 0.4843 -
val_loss: 0.2828 - val_mae: 0.4596
Epoch 6/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.3137 - mae: 0.4782 -
val_loss: 0.2742 - val_mae: 0.4511
Epoch 7/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.3022 - mae: 0.4690 -
val_loss: 0.2650 - val_mae: 0.4417
Epoch 8/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.2904 - mae: 0.4587 -
val_loss: 0.2549 - val_mae: 0.4312
Epoch 9/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.2792 - mae: 0.4500 -
val_loss: 0.2440 - val_mae: 0.4195
Epoch 10/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.2640 - mae: 0.4355 -
val_loss: 0.2325 - val_mae: 0.4067
Epoch 11/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - loss: 0.2526 - mae: 0.4260 -
val_loss: 0.2201 - val_mae: 0.3924
Epoch 12/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 84ms/step - loss: 0.2460 - mae: 0.4161 -
val_loss: 0.2072 - val_mae: 0.3768
Epoch 13/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.2398 - mae: 0.4070 -
val_loss: 0.1940 - val_mae: 0.3600
Epoch 14/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.2153 - mae: 0.3874 -
val_loss: 0.1804 - val_mae: 0.3416
Epoch 15/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.2062 - mae: 0.3745 -
val_loss: 0.1665 - val_mae: 0.3260
Epoch 16/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.1924 - mae: 0.3605 -
val_loss: 0.1525 - val_mae: 0.3116
Epoch 17/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.1718 - mae: 0.3400 -
val_loss: 0.1385 - val_mae: 0.2995
Epoch 18/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.1672 - mae: 0.3351 -
val_loss: 0.1248 - val_mae: 0.2862
```

```
Epoch 19/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.1472 - mae: 0.3125 -
val_loss: 0.1116 - val_mae: 0.2714
Epoch 20/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.1335 - mae: 0.2988 -
val_loss: 0.0995 - val_mae: 0.2610
Epoch 21/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1171 - mae: 0.2858 -
val_loss: 0.0893 - val_mae: 0.2519
Epoch 22/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1104 - mae: 0.2798 -
val_loss: 0.0819 - val_mae: 0.2416
Epoch 23/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.1059 - mae: 0.2772 -
val_loss: 0.0787 - val_mae: 0.2328
Epoch 24/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.0958 - mae: 0.2648 -
val_loss: 0.0808 - val_mae: 0.2329
Epoch 25/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.1022 - mae: 0.2767 -
val_loss: 0.0879 - val_mae: 0.2445
Epoch 26/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1003 - mae: 0.2700 -
val_loss: 0.0968 - val_mae: 0.2621
Epoch 27/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - loss: 0.1428 - mae: 0.3135 -
val_loss: 0.0998 - val_mae: 0.2668
Epoch 28/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.1150 - mae: 0.2756 -
val_loss: 0.0999 - val_mae: 0.2670
Epoch 29/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0985 - mae: 0.2534 -
val_loss: 0.0977 - val_mae: 0.2635
Epoch 30/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0963 - mae: 0.2533 -
val_loss: 0.0946 - val_mae: 0.2581
Epoch 31/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - loss: 0.1017 - mae: 0.2700 -
val_loss: 0.0902 - val_mae: 0.2496
Epoch 32/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - loss: 0.1144 - mae: 0.2816 -
val_loss: 0.0857 - val_mae: 0.2391
Epoch 33/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0908 - mae: 0.2572 -
val_loss: 0.0825 - val_mae: 0.2352
Epoch 34/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.1051 - mae: 0.2754 -
val_loss: 0.0802 - val_mae: 0.2333
Epoch 35/100
```

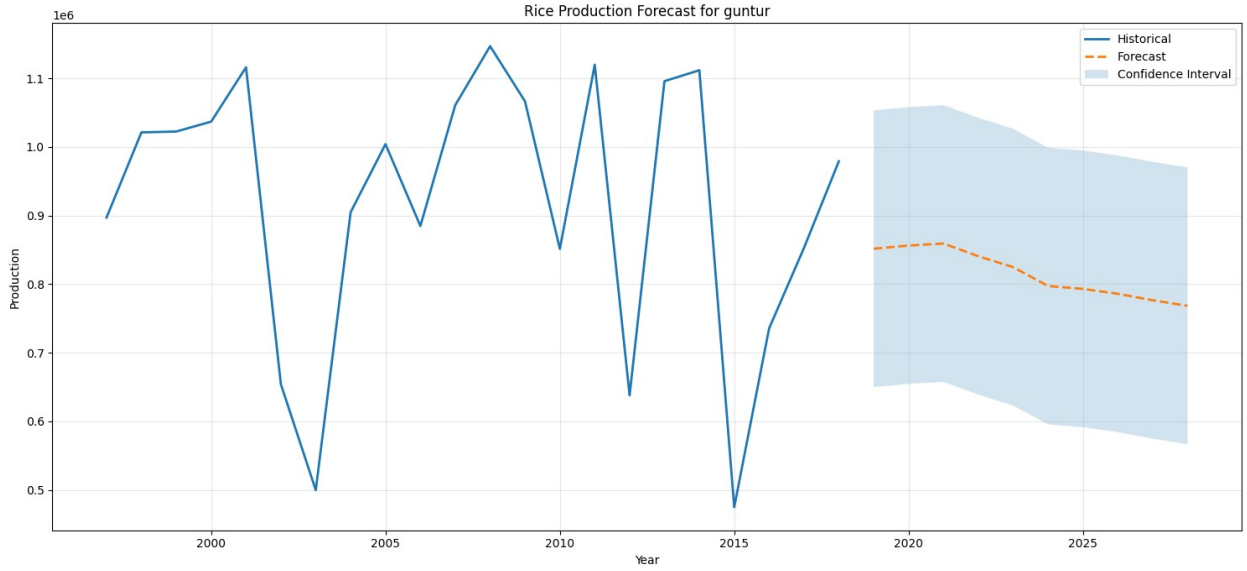
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0942 - mae: 0.2599 -
val_loss: 0.0790 - val_mae: 0.2333
Epoch 36/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - loss: 0.0935 - mae: 0.2550 -
val_loss: 0.0789 - val_mae: 0.2333
Epoch 37/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0937 - mae: 0.2624 -
val_loss: 0.0794 - val_mae: 0.2343
Epoch 38/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0895 - mae: 0.2549 -
val_loss: 0.0802 - val_mae: 0.2373
Epoch 39/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0954 - mae: 0.2597 -
val_loss: 0.0811 - val_mae: 0.2398
Epoch 40/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0954 - mae: 0.2684 -
val_loss: 0.0820 - val_mae: 0.2416
Epoch 41/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0983 - mae: 0.2656 -
val_loss: 0.0826 - val_mae: 0.2427
Epoch 42/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.1021 - mae: 0.2683 -
val_loss: 0.0829 - val_mae: 0.2433
Epoch 43/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0992 - mae: 0.2702 -
val_loss: 0.0829 - val_mae: 0.2434
Epoch 44/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0956 - mae: 0.2621 -
val_loss: 0.0827 - val_mae: 0.2429
Epoch 45/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1071 - mae: 0.2832 -
val_loss: 0.0823 - val_mae: 0.2421
Epoch 46/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0997 - mae: 0.2653 -
val_loss: 0.0816 - val_mae: 0.2408
Epoch 47/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0990 - mae: 0.2696 -
val_loss: 0.0808 - val_mae: 0.2391
Epoch 48/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0979 - mae: 0.2702 -
val_loss: 0.0802 - val_mae: 0.2372
Epoch 49/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0993 - mae: 0.2622 -
val_loss: 0.0795 - val_mae: 0.2349
Epoch 50/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.1042 - mae: 0.2728 -
val_loss: 0.0790 - val_mae: 0.2332
Epoch 51/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.0933 - mae: 0.2604 -
```

```
val_loss: 0.0788 - val_mae: 0.2332
Epoch 52/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.0964 - mae: 0.2674 -
val_loss: 0.0789 - val_mae: 0.2332
Epoch 53/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0994 - mae: 0.2681 -
val_loss: 0.0792 - val_mae: 0.2332
Epoch 54/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0932 - mae: 0.2595 -
val_loss: 0.0796 - val_mae: 0.2330
Epoch 55/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0962 - mae: 0.2555 -
val_loss: 0.0801 - val_mae: 0.2329
Epoch 56/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 0.0882 - mae: 0.2487 -
val_loss: 0.0807 - val_mae: 0.2330
Epoch 57/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0995 - mae: 0.2720 -
val_loss: 0.0811 - val_mae: 0.2334
Epoch 58/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0965 - mae: 0.2667 -
val_loss: 0.0812 - val_mae: 0.2335
Epoch 59/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0956 - mae: 0.2682 -
val_loss: 0.0813 - val_mae: 0.2335
Epoch 60/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0947 - mae: 0.2598 -
val_loss: 0.0810 - val_mae: 0.2332
Epoch 61/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0914 - mae: 0.2541 -
val_loss: 0.0805 - val_mae: 0.2326
Epoch 62/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0977 - mae: 0.2673 -
val_loss: 0.0797 - val_mae: 0.2326
Epoch 63/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0964 - mae: 0.2652 -
val_loss: 0.0791 - val_mae: 0.2326
Epoch 64/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0917 - mae: 0.2552 -
val_loss: 0.0788 - val_mae: 0.2326
Epoch 65/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0992 - mae: 0.2720 -
val_loss: 0.0786 - val_mae: 0.2326
Epoch 66/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0898 - mae: 0.2600 -
val_loss: 0.0785 - val_mae: 0.2326
Epoch 67/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0948 - mae: 0.2601 -
val_loss: 0.0786 - val_mae: 0.2326
```

```
Epoch 68/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0933 - mae: 0.2595 -
val_loss: 0.0788 - val_mae: 0.2331
Epoch 69/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.1002 - mae: 0.2713 -
val_loss: 0.0790 - val_mae: 0.2340
Epoch 70/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0949 - mae: 0.2596 -
val_loss: 0.0792 - val_mae: 0.2348
Epoch 71/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0978 - mae: 0.2636 -
val_loss: 0.0793 - val_mae: 0.2352
Epoch 72/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0958 - mae: 0.2632 -
val_loss: 0.0793 - val_mae: 0.2352
Epoch 73/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0952 - mae: 0.2587 -
val_loss: 0.0791 - val_mae: 0.2348
Epoch 74/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.1004 - mae: 0.2693 -
val_loss: 0.0790 - val_mae: 0.2341
Epoch 75/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0915 - mae: 0.2572 -
val_loss: 0.0788 - val_mae: 0.2333
Epoch 76/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0971 - mae: 0.2654 -
val_loss: 0.0786 - val_mae: 0.2328
Epoch 77/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0904 - mae: 0.2539 -
val_loss: 0.0785 - val_mae: 0.2322
Epoch 78/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.0936 - mae: 0.2626 -
val_loss: 0.0783 - val_mae: 0.2319
Epoch 79/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0883 - mae: 0.2485 -
val_loss: 0.0782 - val_mae: 0.2318
Epoch 80/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0957 - mae: 0.2662 -
val_loss: 0.0782 - val_mae: 0.2317
Epoch 81/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0962 - mae: 0.2660 -
val_loss: 0.0783 - val_mae: 0.2317
Epoch 82/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0941 - mae: 0.2634 -
val_loss: 0.0785 - val_mae: 0.2315
Epoch 83/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0880 - mae: 0.2510 -
val_loss: 0.0788 - val_mae: 0.2314
Epoch 84/100
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0997 - mae: 0.2675 -
val_loss: 0.0791 - val_mae: 0.2313
Epoch 85/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0935 - mae: 0.2605 -
val_loss: 0.0792 - val_mae: 0.2313
Epoch 86/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0965 - mae: 0.2684 -
val_loss: 0.0792 - val_mae: 0.2313
Epoch 87/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0918 - mae: 0.2575 -
val_loss: 0.0793 - val_mae: 0.2313
Epoch 88/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0959 - mae: 0.2569 -
val_loss: 0.0793 - val_mae: 0.2312
Epoch 89/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0913 - mae: 0.2574 -
val_loss: 0.0792 - val_mae: 0.2312
Epoch 90/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0973 - mae: 0.2688 -
val_loss: 0.0792 - val_mae: 0.2312
Epoch 91/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0897 - mae: 0.2541 -
val_loss: 0.0791 - val_mae: 0.2312
Epoch 92/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0941 - mae: 0.2618 -
val_loss: 0.0789 - val_mae: 0.2311
Epoch 93/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0903 - mae: 0.2485 -
val_loss: 0.0787 - val_mae: 0.2311
Epoch 94/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0905 - mae: 0.2564 -
val_loss: 0.0784 - val_mae: 0.2311
Epoch 95/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - loss: 0.0931 - mae: 0.2609 -
val_loss: 0.0782 - val_mae: 0.2311
Epoch 96/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0962 - mae: 0.2619 -
val_loss: 0.0780 - val_mae: 0.2311
Epoch 97/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0942 - mae: 0.2630 -
val_loss: 0.0779 - val_mae: 0.2311
Epoch 98/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0905 - mae: 0.2561 -
val_loss: 0.0779 - val_mae: 0.2310
Epoch 99/100
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0919 - mae: 0.2611 -
val_loss: 0.0778 - val_mae: 0.2310
Epoch 100/100
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - loss: 0.0930 - mae: 0.2595 -
val_loss: 0.0778 - val_mae: 0.2309
```




```
Forecast Values:
Year       Predicted Production
-----------------------------------
2019       851,738.70
2020       856,375.48
2021       859,316.04
2022       840,815.79
2023       824,854.17
2024       797,317.96
2025       793,162.71
2026       786,024.44
2027       776,638.71
2028       768,477.87
```